```
void loop() {
  if (rfid.available()) {
    if (getTag()) printTag();
  }
}

boolean getTag() {
  char startByte = rfid.read();
  delay(20);

  if (startByte == 2) {
    int index = 0;
    while (index < 12) {
      char incomingByte = rfid.read();

      tagString[index] = incomingByte;
      index++;
    }
  }
  rfid.flush();
  return true;
}

void printTag() {
  for (int i=0; i<12; i++) Serial.print(tagString[i]);
  Serial.println(compareTags());
}

const char* compareTags() {
  if (strncmp(tag01, tagString, 12) == 0) return " Tag 1";
  else if (strncmp(tag02, tagString, 12) == 0) return " Tag 2";
  else return " Not recognized.";
}
```

Source Code Summary

To begin with, we need to include the SoftwareSerial library so that we can use it. If we're using a version of the Arduino programming environment prior to 1.0, we need to download and install the library from Mikal Hart's web site (provided later). We then tell the library what to name our instance and what pins we will use. The RFID reader only uses one communication pin, but the library won't let us have an RX pin without a TX pin. We then have a list of known tags that you will definitely want to replace with tags that you own, adding additional arrays as needed. We then set up a place to store the tag that we are currently reading so that we can later compare it to the known tags.

Our normal `setup()` and `loop()` functions are as simple as can be, setting up the data rate for both types of serial, and then in the `loop()` function, checking to see if any data is available. If there is data available, then we make the assumption that a tag has been read. At this point the code calls the `getTag()` function that will be used to read the incoming 16-byte packet for the tag ID. This packet begins with a start byte that is ignored by our function with the the next 12 bytes to be assigned to the character string `tagString[]`. The complete format for this packet is as follows:

```
Start byte (2), 12 bytes (0-9, A-F), Newline (ASCII 10), Return (ASCII 13), End byte (3)
```